How Attackers Compromise Software Projects

Seven common ways trust in packages, accounts, tokens, and automation can be abused.

1 Compromised accounts

Phishing, stolen passwords, email compromise, or stolen tokens can let an attacker impersonate a maintainer.

2 Stolen authentication credentials

npm tokens, GitHub PATs, OAuth tokens, and cloud credentials may provide publishing or repository access.

3 Malicious package updates

With publishing access, an attacker can insert malicious code into a trusted package version, including install-time scripts.

4 Malicious npm packages

Typosquatting and dependency confusion can trick developers into installing an attacker-controlled package.

5 Repository access tokens

A stolen token with write permission can enable code changes, releases, workflow edits, or access to other project resources.

6 GitHub Actions workflows

Unsafe workflows can run untrusted pull-request code with access to secrets or the GITHUB_TOKEN.

7 Spreading between repositories

Malware can search developer machines or CI runners for more credentials, then reuse them to compromise other projects.

KEY IDEA

Attackers target the development process itself, not only the finished application.

Sources: npm Docs · GitHub Docs · GitHub Blog

From “Trusted Package” to Supply-Chain Compromise

Illustrative attack flow and terminal checks.

1. **Trusted npm package**
2. **Developer runs npm install**
3. **Hidden post-install script executes**
4. **npm, GitHub + cloud tokens stolen**
5. **Malicious package versions published**
6. **More developers + repositories infected**

terminal • simulated output

```
$ npm install trusted-package
> trusted-package@2.4.1 postinstall
> node scripts/postinstall.js
! unexpected network activity detected

$ npm audit
1 critical vulnerability

$ git status --short
 M .github/workflows/release.yml

$ npm audit --omit=dev
review production dependency risk

$ gh workflow list
release.yml   active   ← inspect changes
```

Illustrative only: these commands are defensive checks, not proof that a package is safe.

ATTACK CHAIN **Trust → execution → credential theft → propagation**

Sources: npm Docs · GitHub Docs · GitHub Blog